

مقدمه درباره OWASP

OWASP یا "Open Web Application Security Project" یک سازمان غیرانتفاعی جهانی است که هدف اصلی آن، ارتقای امنیت برنامه های وب از طریق ارائه منابع جامع و رایگان است. یکی از مهم ترین دستاوردهای OWASP، فهرست "Top 10" می باشد؛ این فهرست هر چند سال یک بار منتشر می شود و ۱۰ آسیب پذیری رایج و بحرانی برنامه های وب را بر اساس بررسی های جهانی معرفی می کند. آگاهی از این آسیب ها و راهکارهای مقابله با آنها برای همه توسعه دهندگان، مدیران سیستم و کارشناسان امنیت ضروری است؛ چرا که رعایت این موارد، تا حد زیادی حفاظت از داده های کاربران و کاهش حملات سایبری را تضمین می کند.

لیست ۱۰ آسیب پذیری برتر OWASP 2021

۱. Broken Access Control (کنترل دسترسی ناقص)

شرح: زمانی رخ می دهد که کاربران بتوانند به بخش هایی دسترسی پیدا کنند که نباید مجاز باشند (مثلا کاربر عادی به پنل ادمین برسد).

علت: اعمال نادرست یا ناکافی محدودیت ها، اشتباه در سطح دسترسی.

مثال: دسترسی با تغییر پارامتر URL به `admin/``.

۲. Cryptographic Failures (نقص رمزنگاری)

شرح: ضعف یا اشتباه در رمزنگاری داده های حساس مثل پسوردها، کوکی ها یا داده های مالی.

علت: استفاده از الگوریتم های نامناسب (مثلا MD5)، نبود رمزنگاری یا اشتباه در مدیریت کلیدها و salt.

مثال: ذخیره رمز عبور به صورت قابل خواندن یا هش ضعیف.

۳. Injection (تزریق)

شرح: وقوع زمانی که ورودی کاربر مستقیماً در کد یا دیتابیس بدون فیلتر وارد شود و مهاجم بتواند دستورات دلخواه اجرا کند.

علت: فقدان اعتبارسنجی و محافظت در برابر داده های ورودی.

مثال: SQL Injection, Command Injection.

۴. Insecure Design (طراحی ناامن)

شرح: مشکلاتی که در معماری یا طراحی سیستم وجود دارد؛ حتی اگر پیاده سازی درست باشد، اما طراحی اولیه آسیب پذیر است.

مثال: نداشتن احراز هویت دوحله ای، نبود سیاست های امنیتی مناسب.

۵. Security Misconfiguration (پیکربندی اشتباه امنیت)

شرح: تنظیمات اشتباه، پیش فرض یا ناقص در سرور، نرم افزار، سیستم عامل یا سرویس ها که موجب افشای اطلاعات می شود.

مثال: صفحه های خطای مفصل، فعال بودن debugging، پسوردهای پیش فرض.

۶. Vulnerable and Outdated Components (کامپوننت های آسیب پذیر و قدیمی)

شرح: استفاده از کتابخانه ها، ماژول ها یا framework های قدیمی و آسیب پذیر در برنامه.

مثال: استفاده از نسخه قدیمی jQuery یا OpenSSL بدون بروزرسانی.

۷. Identification and Authentication Failures (نقص در شناسایی و احراز هویت)

آزمایش دوم (نقص های رمزنگاری)

فائزه باقریان (کد 1) - حانیه قوی پنجه (کد 3) 4011262321-4011262043

شرح: ضعف در مکانیزم های ورود یا احراز هویت مثل session management یا رمزهای ضعیف.

مثال: نبود محدودیت روی تلاش های ورود، استفاده از رمز عبور ساده.

۸. Software and Data Integrity Failures (نقص در یکپارچگی نرم افزار و داده)

شرح: سوءاستفاده از مواردی که صحت نرم افزارها یا داده ها تضمین نشده است؛ مثلاً بروزرسانی نامعتبر یا تزریق کد مخرب.

مثال: بارگذاری جعبه ابزار یا پلاگین از منابع نامعتبر.

۹. Security Logging and Monitoring Failures (ضعف در ثبت و نظارت امنیتی)

شرح: نبود گزارش گیری یا پایش مناسب وقایع امنیتی که موجب عدم شناسایی حملات می شود.

مثال: عدم ثبت لاگ های login، عدم هشدار در ورودهای مشکوک.

۱۰. Server-Side Request Forgery (SSRF) (جعل درخواست سمت سرور)

شرح: زمانی که سرور به درخواست های جعلی از سمت کاربر پاسخ دهد و محتوای دلخواه مهاجم را بازیابی کند.

مثال: مهاجم سرور را مجبور می کند به URL خاصی درخواست فرستاده و اطلاعات داخلی را در اختیار قرار دهد.

شرح سناریو نقض رمزنگاری

در این پروژه، یک صفحه لاگین ساده طراحی شده است که در آن اطلاعات ورودی کاربر (نام کاربری و رمز عبور) توسط فرم در رجیستر شده است. این برنامه به صورت ناامن نوشته شده است و دارای چندین ضعف مهم است. اولین ضعف، نبود اعتبارسنجی و استفاده نشدن از پارامترهای امن در کوئری های پایگاه داده است، که منجر به بروز

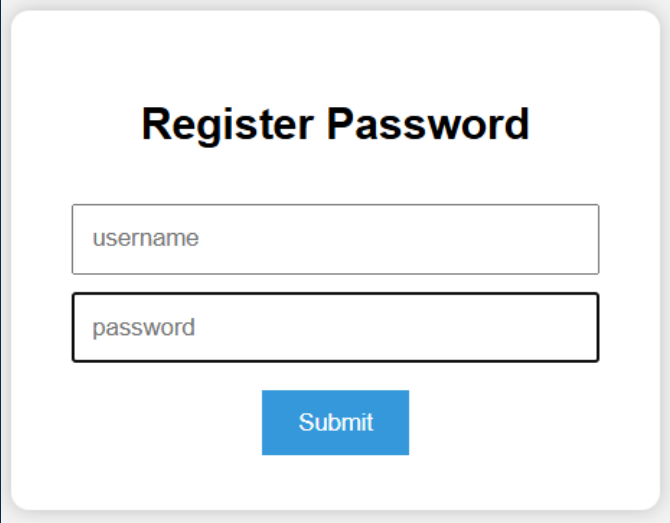
آزمایش دوم (نقص های رمزنگاری)

فائزه باقریان (کد 1) - حانیه قوی پنجه (کد 3) 4011262321-4011262043

آسیب پذیری SQL Injection می شود. در نتیجه مهاجم می تواند با وارد کردن ورودی های خاص در فرم، کنترل برنامه را در دست گیرد و حتی فایل های سرور را دستکاری کند.

نقص دیگر، عدم امنیت در ذخیره سازی رموز عبور است. در این برنامه، رمزها به صورت متن ساده در فایل passwords.txt قرار دارند که به مهاجم امکان دیدن مستقیم رمزهای عبور را می دهد، چیزی که به صورت کلی در برنامه های تولیدی توصیه نمی شود.

در نهایت این طراحی منجر می شود مهاجم بتواند با سوءاستفاده از ضعف های فنی، به فایل های حساس سرور مانند فایل رمزهای عبور (passwords.txt) دسترسی پیدا کند که این مسئله منجر به فاش شدن اطلاعات حساس و نقض امنیت است.



The image shows a web form for registering a password. The form has a title 'Register Password' at the top. Below the title are two input fields: one labeled 'username' and another labeled 'password'. At the bottom of the form is a blue button labeled 'Submit'.

هش ضعیف بدون سالت و تاثیر آن بر امنیت

یکی دیگر از نقاط ضعف مهم در این سناریو، استفاده از الگوریتم هش ضعیف و ناکارآمد است. در این پروژه، فرض بر این است که پسوردهای کاربران به صورت هش MD5 رمزگذاری شده اند، بدون اینکه از تکنیک های دیگر مانند salt استفاده شده باشد.

آزمایش دوم (نقص های رمزنگاری)

فائزه باقریان (کد 1) - حانیه قوی پنجه (کد 3) 4011262321-4011262043

این نوع هش گذاری بسیار آسیب پذیر است، زیرا الگوریتم MD5 در مقابل حمله های Collision و Rainbow Table کاملاً شکست پذیر است و به راحتی می توان هش ها را کرک کرد. تلاش مهاجم برای تجزیه هش های ضعیف و بازیابی رمزهای اصلی، در مدت زمان کوتاهی انجام پذیر است، که در نتیجه امنیت کاربران کاملاً زیر سوال می رود.

این اشکال ناشی از طراحی ناامن است که سرور بدون استفاده از امکانات حفاظتی، رمز عبورهای کاربر را در قالب هش های قابل کرک قرار می دهد، و مهاجم پس از دسترسی به فایل passwords.txt، خیلی سریع به رمزهای واقعی دست می یابد.

App.py

```
C:\crypto-lab>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production de
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 684-821-596
127.0.0.1 - - [03/May/2026 14:44:01] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [03/May/2026 14:44:01] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [03/May/2026 14:44:23] "POST /save HTTP/1.1" 200 -
* Detected change in 'C:\\crypto-lab\\cracker.py', reloading
* Restarting with stat
* Debugger is active!
* Debugger PIN: 684-821-596
```

Passwords.txt

File	Edit	View
<u>faeze</u> : f2abb12160bd192c970d892f182fd677		
<u>hanie</u> : 4297f44b13955235245b2497399d7a93		

سناریو حمله

در این سناریو، برنامه وب به شکل ناامن تنظیم شده است که مهاجم می تواند با exploiting آسیب پذیری Local File Inclusion یا Direct File Reading، به سادگی محتویات فایل passwords.txt را مشاهده کند.

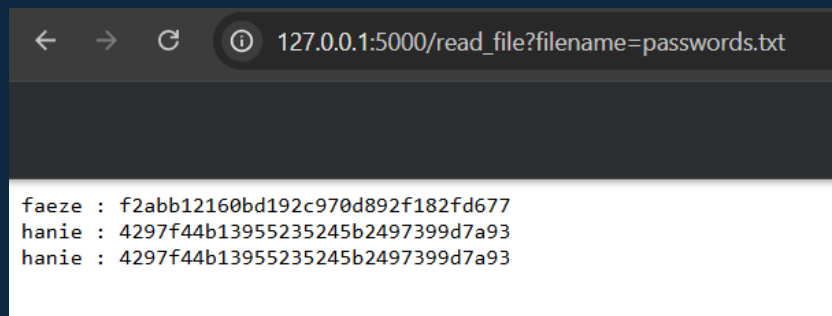
مهاجم پس از پیدا کردن این آسیب پذیری، وارد آدرس زیر در مرورگر می شود:

آزمایش دوم (نقص های رمزنگاری)

فائزه باقریان (کد 1) - حانیه قوی پنجه (کد 3) 4011262321-4011262043

http://localhost:5000/read_file?filename=passwords.txt

در این حالت، سرور فایل passwords.txt را خوانده و محتویات آن را در خروجی نمایش می دهد. این نتیجه، فاش شدن اطلاعات حساس است و نشان می دهد که سیستم به شدت در برابر چنین حملات ساده ای آسیب پذیر است. در این فایل، رمزهای عبور به صورت متنی و قابل دیدن است که مهاجم می تواند آن ها را استخراج کرده و در ادامه آنها را برای کرک کردن هش ها استفاده کند.



```
← → ↻ ⓘ 127.0.0.1:5000/read_file?filename=passwords.txt  
faeze : f2abb12160bd192c970d892f182fd677  
hanie : 4297f44b13955235245b2497399d7a93  
hanie : 4297f44b13955235245b2497399d7a93
```

نقش ابزار کرک و فرآیند کرک کردن رموز

پس از اینکه مهاجم فایل passwords.txt حاوی رمزهای عبور را دریافت کرد مرحله بعدی استفاده از ابزارهای کرک است. مهاجم می تواند مستقیماً از سایت های کرک مثل <https://crackstation.net> یا استفاده از ورد دیکشنری <https://freerainbowtables.com/en/tables2> اقدام به کرک رمز ها بکند (که برای ما فیلتر است) یا ابتدا هش های موجود در فایل را استخراج کند و سپس با استفاده از اسکریپت های مانند cracker.py فرآیند کرک کردن را آغاز کند. در اینجا هم به یک ورد دیکشنری نیاز داریم که شامل لیستی از رمز های رایج است یا مهاجم می تواند طبق شناختی که از قربانی دارد لیست را پر کند و یا از ورد دیکشنری های آماده استفاده کند.

آزمایش دوم (نقص های رمزنگاری)

فائزه باقریان (کد 1) - حانیه قوی پنجه (کد 3) 4011262043-4011262321

مثال یک ورد دیکشنری در سناریوی ما

File	Edit	View
123456		
password		
admin		
qwerty		
12345678		
111111		
abc123		
<u>iloveyou</u>		
welcome		
123123		
1		

در این فرآیند هش های MD5 که معمولا در فایل ذخیره شده است به کمک wordlist تست می شود. اگر هش ضعیف باشد، رمز به سرعت بازیابی می شود.

```
C:\crypto-lab>python cracker.py
Enter MD5 hash: 4297f44b13955235245b2497399d7a93
[*] Starting dictionary attack...
[+] Password found: 123123
```

در نتیجه مهاجم می تواند رمزهای واقعی کاربران را در کوتاه ترین زمان ممکن بدست آورد و این نشان دهنده اهمیت نگهداری امن و معتبر رمزها است.

خلاصه آسیب پذیری ها

در این سناریو چند آسیب پذیری اصلی مشاهده شد که امنیت برنامه را به خطر انداخته است. استفاده از الگوریتم هش ضعیف بدون salt، نبود اعتبارسنجی مناسب در فایل های ورودی، و طراحی ناپایدار برنامه منجر به فاش شدن اطلاعات حساس شده است. از نظر سطح امنیت، این طراحی نه تنها مقاومت کافی در برابر حملات ندارند، بلکه امکاناتی فراهم

آزمایش دوم (نقص های رمزنگاری)

فائزه باقریان (کد 1) - حانیه قوی پنجه (کد 3) 4011262321-4011262043

کرده است که مهاجم بتواند به سادگی به داده های حساس دسترسی پیدا کند، فایل های سرور را بخواند و در ادامه رمزهای عبور کاربران را کرک کند. این مشکلات باید در مراحل بعدی با روش های مناسب جلوگیری و اصلاح شوند تا برنامه امن تر و مقاوم تر گردد.

راهکار های رمزنگاری امن

راهکار اول: استفاده از الگوریتم های هشینگ مقاوم و مدرن

اولین قدم برای اصلاح، جایگزینی الگوریتم ضعیف MD5 با یک الگوریتم مقاوم در برابر حملات جستجوی فراگیر (Brute-force) مانند bcrypt یا Argon2 است. این الگوریتم ها به گونه ای طراحی شده اند که از نظر محاسباتی سنگین باشند، بنابراین سرعت تست کردن رمزها توسط مهاجم به شدت کاهش می یابد. برخلاف MD5 که برای سرعت بالا طراحی شده، bcrypt زمان لازم برای تولید هر هش را افزایش می دهد که این امر کرک کردن آن را عملاً غیرممکن یا بسیار هزینه بر می کند.

راهکار دوم: استفاده از Salt تصادفی و منحصر به فرد

برای جلوگیری از حملات Rainbow Table و تشخیص رمزهای مشابه، باید از تکنیک Salt استفاده کرد. Salt یک رشته متنی کاملاً تصادفی است که قبل از عملیات هشینگ به رمز عبور کاربر اضافه می شود. با این کار، حتی اگر دو کاربر رمز عبور یکسانی (مثلاً 123456) داشته باشند، به دلیل متفاوت بودن Salt آن ها، هش های ذخیره شده در فایل نهایی کاملاً متفاوت خواهند بود. این موضوع باعث می شود مهاجم نتواند با داشتن یک لیست از هش های آماده، رمزها را تشخیص دهد.

راهکار سوم: اصلاح دسترسی به فایل و اعتبارسنجی ورودی ها

برای جلوگیری از دسترسی مستقیم مهاجم به فایل رمزها، باید دو اقدام اساسی انجام شود. اول، حذف تابعی که به صورت مستقیم نام فایل را از ورودی URL می گیرد و آن را باز می کند. دوم، محدود کردن سطح دسترسی فایل

آزمایش دوم (نقص های رمزنگاری)

فائزه باقریان (کد 1) - حانیه قوی پنجه (کد 3) 4011262321-4011262043

passwords.txt در سیستم عامل به طوری که فقط یوزر اجرا کننده سرویس پایتون اجازه خواندن آن را داشته باشد. همچنین نباید اجازه داده شود که نام فایل ها به صورت مستقیم در پارامترهای وب ارسال شوند.

بخش اصلاحات

پس از شناسایی نقاط ضعف در سیستم اولیه، مجموعه ای از اصلاحات ساختاری برای تطبیق با استانداردهای OWASP اعمال شد. هدف اصلی این اصلاحات، جلوگیری از افشای اطلاعات در صورت دسترسی غیرمجاز به فایل های سرور و همچنین مقاوم سازی سیستم در برابر حملات دیکشنری و بروتفورس است. این تغییرات شامل به روزرسانی الگوریتم های رمزنگاری و مدیریت دسترسی ها می باشد.

```
def hash_password(password):  
    salt = bcrypt.gensalt()  
    hashed = bcrypt.hashpw(password.encode('utf-8'), salt)  
    return hashed
```

در نسخه اصلاح شده، الگوریتم MD5 به طور کامل حذف و کتابخانه bcrypt جایگزین آن شد. به صورت خودکار فرآیند ایجاد Salt را مدیریت می کند و آن را در داخل خود رشته هش ذخیره می نماید. این الگوریتم از یک هزینه محاسباتی (Work Factor) استفاده می کند که باعث می شود عملیات بررسی هر رمز عبور به زمان مشخصی (مثلاً ۱۰۰ میلی ثانیه) نیاز داشته باشد. این تأخیر ناچیز برای کاربر معمولی، برای مهاجمی که قصد دارد میلیاردها رمز را در ثانیه تست کند، مانعی عبورناپذیر ایجاد می کند.

با پیاده سازی Salt تصادفی برای هر کاربر، امنیت داده ها به طور چشمگیری افزایش یافته است. در این حالت، حتی اگر مهاجم موفق به دریافت فایل passwords.txt شود، دیگر نمی تواند از جدول های پیش محاسبه شده (Rainbow Tables) استفاده کند. زیرا هر هش منحصر به فرد است و مهاجم ناچار است برای هر کاربر به صورت جداگانه عملیات کرک را انجام دهد که با توجه به کند بودن bcrypt، این فرآیند سال ها به طول خواهد انجامید.

آزمایش دوم (نقص های رمزنگاری)

فائزه باقریان (کد 1) - حانیه قوی پنجه (کد 3) 4011262321-4011262043

Passwords-safe.txt

File Edit View

faeze:\$2b\$12\$cnBYvLa0X0NoH0kd3H/EP0P8UUcAdS7opQkZdat5c8x.4QzxgK2Y0

در پایان این مرحله، برنامه در برابر نقص های Cryptographic Failures مقاوم شده است. با استفاده از تکنیک های مدرن هشینگ و جلوگیری از دسترسی های مستقیم به فایل، زنجیره حمله مهاجم قطع گردید. حتی در صورت لو رفتن فایل حاوی هش ها، به دلیل قدرت بالای الگوریتم استفاده شده و وجود Salt، امکان بازیابی رمزهای اصلی توسط ابزارهایی مانند [cracker.py](#) وجود نخواهد داشت. این رویکرد تضمین می کند که امنیت کاربران بر پایه مخفی نگه داشتن فایل ها نیست، بلکه بر پایه قدرت ریاضی الگوریتم های رمزنگاری بنا شده است.